ELIDIA

Un videogioco Arcade per l'apprendimento delle nozioni base di armonia musicale
Progetto e documentazione tecnica

Campionamento e Sintesi III

Ottobre 2026



Indice

1 Introduzione **3**

2 Obiettivo didattico **3**

3 Gameplay e regole **3**

3.1 Scenario ed interazione 3

3.2 Progressione della modalità "Intermedia" 4

3.3 Introduzione alla teoria 4

4 Architettura generale **4**

5 Modulo audio **4**

5.1 Architettura 4

5.2 Generazione dell'altezza mediante interpolazione lineare 5

5.3 Pre-calcolo delle altezze 5

5.4 Modifica dell'ampiezza del sample: `sample_energy()` 5

5.5 Variazione della curva d'attacco: `sample_attack()` 6

5.6 Catena dei processi e polifonia 6

6 Nemici come strutture di dati **6**

6.1 La lista `enemy_components` 6

6.2 Direzioni, movimento e collisioni 7

6.3 Bersaglio più vicino e mirino 7

7 Generazione e riconoscimento di intervalli **7**

7.1 Rappresentazione cromatica 7

7.2 Intervalli melodici: una piccola macchina a stati 8

7.3 Bicordi 8

8 MIDI Input e Game-Loop **8**

9 Limiti attuali e sviluppi futuri **9**

10 Conclusioni **9**

1 Introduzione

ELIDIA è un videogioco sviluppato in Python utilizzando la libreria Pygame. Il progetto nasce dall'intento di trasformare la pratica del riconoscimento e dell'esecuzione di note musicali in un'attività interattiva: l'esecuzione musicale non si limita ad accompagnare l'azione, ma costituisce il comando necessario per affrontare i nemici.

Il suo stile visivo richiama i tipici videogiochi arcade anni 90', mentre le sue regole si basano su concetti base di armonia musicale . Il personaggio principale (Elidia) occupa il centro dello spazio di gioco; i nemici, rappresentati come note musicali ostili, compaiono ai bordi dello schermo e avanzano verso di lei. Per eliminarli, il giocatore deve eseguire le note indicate utilizzando una tastiera MIDI.

La presente documentazione si concentra principalmente sulla modalità *"Intermedia"*, che costituisce l'oggetto della presentazione d'esame. Le modalità "Facile" e "Difficile" fanno parte del progetto complessivo, ma non sono da considerarsi complete né necessarie ai fini della valutazione di questa versione.

2 Obiettivo didattico

L'obiettivo della modalità "Intermedia" è esercitarsi a riconoscere ed eseguire intervalli musicali attraverso un sistema di feedback immediato. Un intervallo è la relazione di altezza tra due suoni; può essere descritto mediante il numero di gradi della scala diatonica e la distanza cromatica in semitoni.

Questa modalità si articola in due fasi: durante le prime cinquanta eliminazioni di nemici, il giocatore esegue le note *in successione*, rispettando l'ordine indicato (in questa versione Beta troveremo solo intervalli ascendenti); successivamente, le note devono essere suonate come un *accordo di due note*, con attacchi quasi simultanei. Il gioco rende così evidente la distinzione tra intervalli melodici e armonici.

La pressione temporale ed il "rewarding" introducono una sfida divertente, ma l'obiettivo didattico rimane l'associazione tra simboli musicali, altezza dei suoni ed esecuzione fisica.

ELIDIA non intende sostituire l'insegnamento della teoria musicale o dell'educazione dell'orecchio, si tratta piuttosto di uno strumento di esercitazione complementare game-based.

3 Gameplay e regole

3.1 Scenario ed interazione

Il gioco si svolge all'interno di un'area di 1920×1080 pixel, adattata alle dimensioni della finestra. La protagonista rimane al centro, mentre i nemici compaiono casualmente lungo uno dei quattro bordi e avanzano verso di lei. Per evitare sovraffollamento e garantire la leggibilità dello spazio di gioco, non possono essere presenti contemporaneamente più di sei nemici.

Il sistema individua il nemico più vicino e lo evidenzia con un mirino. Il bersaglio viene aggiornato costantemente nel corso della partita; al di sotto della threshold delle cinquanta eliminazioni, quindi nello stato delle successioni di due note, una volta che il giocatore ha eseguito la prima nota di un intervallo, il riferimento a quel nemico viene mantenuto finché il tentativo non viene completato o annullato. Ciò evita che il superamento di un nemico rispetto a quello attualmente selezionato, possa alterare arbitrariamente l'esercizio musicale in corso con conseguente stato di frustrazione nell'utente. Il giocatore può quindi scegliere se giocare con una tastiera

MIDI oppure con la tastiera del proprio Mac. All'avvio, viene visualizzato un menù con i vari dispositivi di input MIDI collegati alla macchina. Le note vengono convertite in stringhe come `C3`, `F#3` e `C4`, utilizzate da entrambi i sistemi di input.

3.2 Progressione della modalità "Intermedia"

Ogni nemico porta con sé due note. Fino a 49 eliminazioni, viene visualizzata inoltre sulla sua testa una freccia, ad esempio `C3 -> E3`, per indicare una sequenza ordinata; la prima nota suonata correttamente viene evidenziata in verde. A partire dalla cinquantesima eliminazione, la coppia viene visualizzata come una tupla, ad esempio `('C3', 'E3')`, per indicare un accordo di due note. La finestra di tolleranza tra gli attacchi è impostata a circa 100 millisecondi.

Il contatore delle eliminazioni consente al giocatore di visualizzare costantemente i propri progressi. Un'esecuzione corretta elimina il nemico e genera un piccolo effetto grafico «+1» che sale in dissolvenza verso l'alto per poi svanire. A 100 eliminazioni il livello viene completato e la partita termina con una vittoria. Se un nemico raggiunge la protagonista, le sottrae un punto salute e scompare. In questa versione Beta realizzata per l'esame, la salute iniziale è pari a 100 punti e la partita termina quando questa si esaurisce.

3.3 Introduzione alla teoria

Prima dell'inizio della modalità "Intermedia", una schermata introduttiva definisce cos'è l'intervallo musicale e spiega la differenza tra esecuzione melodica e accordo di due note. Il gioco ha inizio solo dopo la conferma da parte del giocatore.

4 Architettura generale

Il programma è strutturato attorno a un *game loop* a 60 FPS. Ogni iterazione gestisce eventi da tastiera e MIDI, la creazione di elementi, il movimento, le collisioni, la selezione dei bersagli, la verifica musicale e, infine, il rendering. La frequenza dei fotogrammi è controllata tramite `clock.tick(FPS)`.

La separazione logica di queste fasi è importante poiché il codice aggiorna innanzitutto lo stato del mondo di gioco e solo successivamente lo visualizza. Moduli esterni gestiscono, rispettivamente, l'elaborazione dell'audio e la comunicazione MIDI, mentre il file principale coordina le regole del gioco.

5 Modulo audio

5.1 Architettura

L'elaborazione del suono è implementata in `Modulo_audio.py` utilizzando Pygame e NumPy. Invece di utilizzare un campione registrato separatamente per ogni altezza tonale, ELIDIA parte da un unico campione di un suono di una corda pizzicata di un'arpa sulla nota `C3`. La funzione `pitch_sample()` genera le altre altezze ricampionando il campione. Due funzioni aggiuntive, `sample_energy()` e `sample_attack()`, modificano il suono risultante in base al valore della salute della protagonista.

Pygame converte i suoni in array utilizzando `pygame.sndarray.array()` e ricostruisce suoni riproducibili mediante `pygame.sndarray.make_sound()`. NumPy esegue l'interpolazione e la moltiplicazione campione per campione.

5.2 Generazione dell'altezza mediante interpolazione lineare

Per eseguire una trasposizione di n semitoni nel sistema temperato equabile, il rapporto è

$$r(n) = 2^{n/12}. \tag{1}$$

Una quinta giusta sopra la nota di partenza ($n = 7$) dà $r \approx 1.498$, mentre un'ottava ($n = 12$) dà $r = 2$.

Il suono originale è convertito in un array di N_{original} campioni. L'algoritmo determina la lunghezza dell'output come

$$N_{\text{new}} = \left\lfloor \frac{N_{\text{original}}}{r} \right\rfloor. \tag{2}$$

Viene quindi costruito un insieme uniformemente spaziato di N_{new} posizioni che coprono la forma d'onda originale e usato `numpy.interp()` per calcolare l'ampiezza in ciascuna posizione. L'interpolazione lineare stima i valori del campione intermedio dai valori vicini nel segnale originale.

La funzione supporta array sia mono che stereo. Per un campione mono (un array unidimensionale), l'interpolazione viene eseguita una volta. Per quello stereo invece (un array bidimensionale), i canali sinistro e destro vengono interpolati indipendentemente e poi ricombinati:

```
if original.ndim == 1:
    resampled = np.interp(new_positions, old_positions, original)
elif original.ndim == 2:
    left = np.interp(new_positions, old_positions, original[:, 0])
    right = np.interp(new_positions, old_positions, original[:, 1])
    resampled = np.column_stack((left, right))
```

Poiché questo approccio modifica il numero di campioni mantenendo invariata la frequenza di campionamento di riproduzione, *l'altezza e la durata variano insieme.* Innalzare l'altezza accorcia il suono; abbassarla lo allunga. Si tratta di un semplice ricampionamento, non di una variazione dell'altezza con conservazione della durata né di time stretching.

5.3 Pre-calcolo delle altezze

Il programma principale associa i nomi delle note a scostamenti in semitoni rispetto al `C3`. I suoni trasposti vengono generati all'avvio e memorizzati nel dizionario `harp_sounds`. Ciò significa che il ricampionamento dell'altezza non viene ripetuto a ogni attacco di nota durante il gioco.

```
harp_sounds = {}
for note in notes:
    harp_sounds[note] = pitch_sample(sample, note_semitones[note])
for note in altered_notes:
    harp_sounds[note] = pitch_sample(sample, note_semitones[note])
```

5.4 Modifica dell'ampiezza del sample: `sample_energy()`

La funzione `sample_energy(sound, lives)` modifica l'ampiezza complessiva del segnale moltiplicando ogni campione per un coefficiente a. Il coefficiente viene scelto in questo modo:

$$a(L) = \begin{cases} 1.0 & L \geq 50, \\ 0.7 & 20 \leq L < 50, \\ 0.4 & 10 \leq L < 20, \\ 0.25 & L < 10. \end{cases} \tag{3}$$

Per ogni campione $x[k]$, il valore elaborato è

$$y[k] = a(L)x[k]. \tag{4}$$

5.5 Variazione della curva d'attacco: `sample_attack()`

La funzione `sample_attack(sound, lives)` modifica l'attacco del suono mediante un breve *fade-in* lineare. La durata dell'attacco viene scelta casualmente in base ai seguenti intervalli:

$$T_{\text{attack}}(L) = \begin{cases} 0\text{–}5 \text{ ms} & L \geq 50, \\ 10\text{–}15 \text{ ms} & 20 \leq L < 50, \\ 15\text{–}20 \text{ ms} & L < 20. \end{cases} \tag{5}$$

La durata viene convertita in un numero di campioni utilizzando la frequenza di campionamento f_s del mixer:

$$N_{\text{attack}} = \left\lfloor f_s \frac{T_{\text{attack}}}{1000} \right\rfloor. \tag{6}$$

La funzione genera un inviluppo utilizzando `numpy.linspace(0, 1, N_attack)` e ad esso moltiplica i campioni iniziali:

$$y[k] = x[k]e[k] \quad \text{durante l'intervallo di attacco.} \tag{7}$$

I campioni rimanenti rimangono invariati. Per i segnali stereo, lo stesso inviluppo viene applicato a entrambi i canali, preservandone l'equilibrio relativo. Una variazione casuale fa sì che l'attacco di ogni nota non abbia sempre esattamente la stessa durata.

5.6 Catena dei processi e polifonia

Durante il gioco, una nota riconosciuta, proveniente dalla tastiera o dal protocollo MIDI, richiama il relativo suono d'arpa precalcolato. Il gioco applica innanzitutto una scalatura dell'ampiezza, seguita dall'inviluppo di attacco, per poi riprodurre il risultato così elaborato:

```
processed_sound = sample_energy(harp_sounds[played_note], hero_life)
processed_sound = sample_attack(processed_sound, hero_life)
processed_sound.play()
```

La catena di elaborazione è quindi:

Sample arpa (C3) $\longrightarrow$ pitch resampling $\longrightarrow$ riscalatura dell'ampiezza $\longrightarrow$ attack envelope $\longrightarrow$ playback.

Il mixer di Pygame è configurato a 32 canali, il che consente di generare polifonia.

6 Nemici come strutture di dati

6.1 La lista `enemy_components`

Ogni nemico è rappresentato come una lista contenente quattro componenti:

```
enemy_components = [enemy_rect, enemy_note, enemy_position, direction]
enemies.append(enemy_components)
```

Elemento	Scopo
`enemy_rect`	Rettangolo utilizzato per generare l'area del nemico e il rilevamento delle collisioni.
`enemy_note`	Informazioni musicali; in modalità intermedia, una tupla di due note.
`enemy_position`	Posizione in virgola mobile memorizzata come `pygame.Vector2`.
`direction`	vettore unitario che definisce la direzione del movimento verso l'eroina.

La lista `enemies` contiene tutti i nemici presenti nello spazio di gioco. Nuovi elementi vengono aggiunti, quelli esistenti vengono aggiornati tramite un ciclo `for`, mentre i nemici sconfitti o che hanno colliso con il rettangolo dell'eroina vengono rimossi. Il rettangolo e la posizione a virgola mobile assolvono a scopi differenti: quest'ultima consente di mantenere i movimenti frazionari quando la velocità non è un numero intero.

6.2 Direzioni, movimento e collisioni

Quando un nemico viene generato, viene calcolato e normalizzato il vettore che parte dalla sua posizione al centro dell'eroina. Ad ogni frame, la sua posizione viene aggiornata secondo

$$\mathbf{p}_{t+1} = \mathbf{p}_t + v\widehat{\mathbf{d}}, \tag{8}$$

dove v è la velocità di movimento in pixel per frame e $\widehat{\mathbf{d}}$ è il vettore unitario di direzione. Le coordinate del rettangolo vengono aggiornate arrotondando la posizione in virgola mobile. Le collisioni con l'eroina vengono rilevate utilizzando `colliderect()`.

La normalizzazione rende la velocità confrontabile per i nemici che compaiono su lati diversi, sebbene non garantisca tempi di percorrenza identici: la distanza dall'eroina dipende infatti dalla posizione in cui si generano.

6.3 Bersaglio più vicino e mirino

Il programma calcola la distanza euclidea di ciascun nemico dal centro dell'eroina:

$$d = \sqrt{(x_e - x_h)^2 + (y_e - y_h)^2}. \tag{9}$$

Il riferimento al nemico che si trova alla distanza minore dal centro viene conservato in `nearest_enemy`. Un mirino rende la selezione visibile al giocatore. Durante lo stato relativo alla successione delle note, `interval_target` mantiene il riferimento al nemico associato alla prima nota eseguita correttamente.

7 Generazione e riconoscimento di intervalli

7.1 Rappresentazione cromatica

Il dizionario `note_semitones` associa le note disponibili a numeri interi compresi tra 0 e 12, dove il Do3 (C3) corrisponde a 0 e il Do4 (C4) a 12. Un secondo dizionario, `interval_semitones`, associa le denominazioni degli intervalli — come `2m`, `3M` e `5G` — alla loro distanza cromatica.

Per creare un nemico, il programma sceglie una nota iniziale e un intervallo, quindi calcola

$$s_2 = s_1 + \Delta s. \tag{10}$$

Se il risultato eccede il registro disponibile, viene estratta un'altra combinazione. In caso contrario, la seconda nota viene determinata a partire dal suo scostamento in semitoni e la coppia viene memorizzata come `(prima_nota, seconda_nota)`. Questa rappresentazione rende esplicita la distanza cromatica e mantiene gli intervalli generati entro l'estensione di note massima.

7.2 Intervalli melodici: una piccola macchina a stati

Durante le prime quarantanove eliminazioni, l'ordine delle note è importante. La logica di riconoscimento utilizza `interval_progress`: lo stato 0 è in attesa della prima nota, mentre lo stato 1 è in attesa della seconda. Se la prima nota è corretta, viene memorizzato `interval_target` e il progresso viene indicato visivamente. Se anche la seconda nota è corretta, il nemico viene eliminato e il contatore incrementato; se invece la seconda nota è errata, il tentativo viene azzerato.

Stato	Evento	Risultato
0	Prima nota corretta	Inizio stato 1; target memorizzato
0	Prima nota errata	Stato 0
1	Seconda nota corretta	Nemico eliminato; stato 0
1	Seconda nota errata	Cancella tentativo; ritorno allo stato 0

7.3 Bicordi

A partire dalla cinquantesima eliminazione, le stesse coppie di note diventano bicordi. Il loro ordine di esecuzione non è vincolato. La prima nota corretta avvia un tentativo e fa partire un counter `interval_start_time`. La seconda deve appartenere alla coppia ed essere diversa dalla prima. Il requisito temporale è

$$t_2 - t_1 \leq 100 \text{ ms}. \tag{11}$$

Se la seconda nota arriva troppo tardi o è errata, il tentativo viene annullato. Si tratta di una *definizione operativa di simultaneità* basata su un onset, anziché su una misurazione continua della sovrapposizione tra i tasti.

Il lettore dei messaggi MIDI restituisce un elenco di attacchi di nota ricevuti all'interno di un singolo frame. Questo aspetto è importante poiché entrambe le note di un bicordo possono arrivare durante lo stesso aggiornamento: elaborare solo l'ultima nota impedirebbe di riconoscere molte esecuzioni simultanee.

8 MIDI Input e Game-Loop

Il modulo MIDI utilizza `pygame.midi` per elencare i dispositivi di input, aprire quello selezionato dall'utente e leggere gli eventi senza bloccare il ciclo principale. Se non viene selezionato alcun dispositivo, la tastiera del computer rimane utilizzabile.

I messaggi MIDI di tipo *Note On* con una *velocity* superiore a zero indicano l'inizio di una nota. Il numero della nota MIDI viene convertito nel nome della nota musicale tramite una tabella

delle classi di altezza (basata su dodici semitoni) e un calcolo dell'ottava. La nota MIDI 48 corrisponde al Do3 (C3) e la nota 60 al Do4 (C4). La tastiera del computer utilizza la stessa notazione testuale per le note.

Questa normalizzazione consente al codice di riconoscimento musicale di funzionare con controller differenti. Inoltre, permette una separazione delle responsabilità: il modulo MIDI interpreta i messaggi esterni, mentre il programma principale stabilisce se le note eseguite soddisfino le regole del gioco.

9 Limiti attuali e sviluppi futuri

La versione attuale di ELIDIA si concentra principalmente sulla modalità Intermedia, sviluppata e resa pienamente funzionante per consentire una valutazione concreta delle meccaniche di gioco e delle soluzioni tecniche adottate.

Gli sviluppi futuri riguarderanno innanzitutto l'ampliamento e il perfezionamento del percorso didattico, attraverso una progressione più strutturata degli intervalli musicali. In particolare, l'introduzione di intervalli discendenti e di un sistema di generazione che proponga inizialmente intervalli costruiti sulle note della scala di Do maggiore, per poi estendersi gradualmente a combinazioni cromatiche e casuali.

Sul piano tecnico invece, sarà necessario perfezionare il bilanciamento del movimento dei nemici, tenendo conto delle diverse distanze che questi devono percorrere in funzione del punto di spawn. È inoltre previsto il completamento delle modalità Facile e Difficile, dove quest'ultima sarà dedicata allo studio e al riconoscimento delle triadi maggiori, minori, aumentate e diminuite.

Per quanto riguarda l'esperienza di gioco, gli sviluppi futuri vedranno l'introduzione di un tutorial e l'arricchimento delle meccaniche di gioco attraverso sistemi di combo, bonus, malus e abilità speciali (perk), con l'obiettivo di rendere il gameplay più dinamico e coinvolgente.

Infine, sarà necessario sviluppare una componente narrativa di ispirazione fiabesca, in modo da conferire maggiore identità all'universo di ELIDIA e di aiutare il giocatore ad immergersi nel mondo di gioco, integrando l'apprendimento musicale all'interno di un percorso narrativo coerente.

10 Conclusioni

ELIDIA unisce programmazione, interazione in tempo reale ed educazione musicale. Il suo tratto distintivo risiede nell'integrazione della teoria musicale nella meccanica di gioco fondamentale: riconoscere ed eseguire un intervallo non costituisce un esercizio a sé stante, bensì l'azione necessaria per avanzare nel gioco.

La gestione delle liste di nemici, i vettori di movimento, la generazione di intervalli cromatici, la macchina a stati per il riconoscimento e la normalizzazione degli input MIDI mostrano come concetti di programmazione possano combinarsi per dare vita a un'esperienza musicale videoludica.